DWEC · 2º DAW · UNIDAD 6

Práctica: TecnoShop

Una tienda de ordenadores con clases, herencia, eventos y DOM

Entrega

En la tarea indica **dos URLs**: la del repositorio de GitHub y la de la página publicada (GitHub Pages).

Qué vas a construir

Vas a programar **TecnoShop**, la página de una tienda que vende ordenadores de sobremesa y portátiles. Tendrá un **catálogo** de productos que se puede filtrar y ordenar, un **carrito** en el que se añaden y quitan productos con su total, y un **formulario** para dar de alta productos nuevos.

Todo el funcionamiento se apoya en tres clases: `Ordenador`, `Portatil` (que hereda de `Ordenador`) y `Carrito`. El diseño es libre; la imagen es solo orientativa.

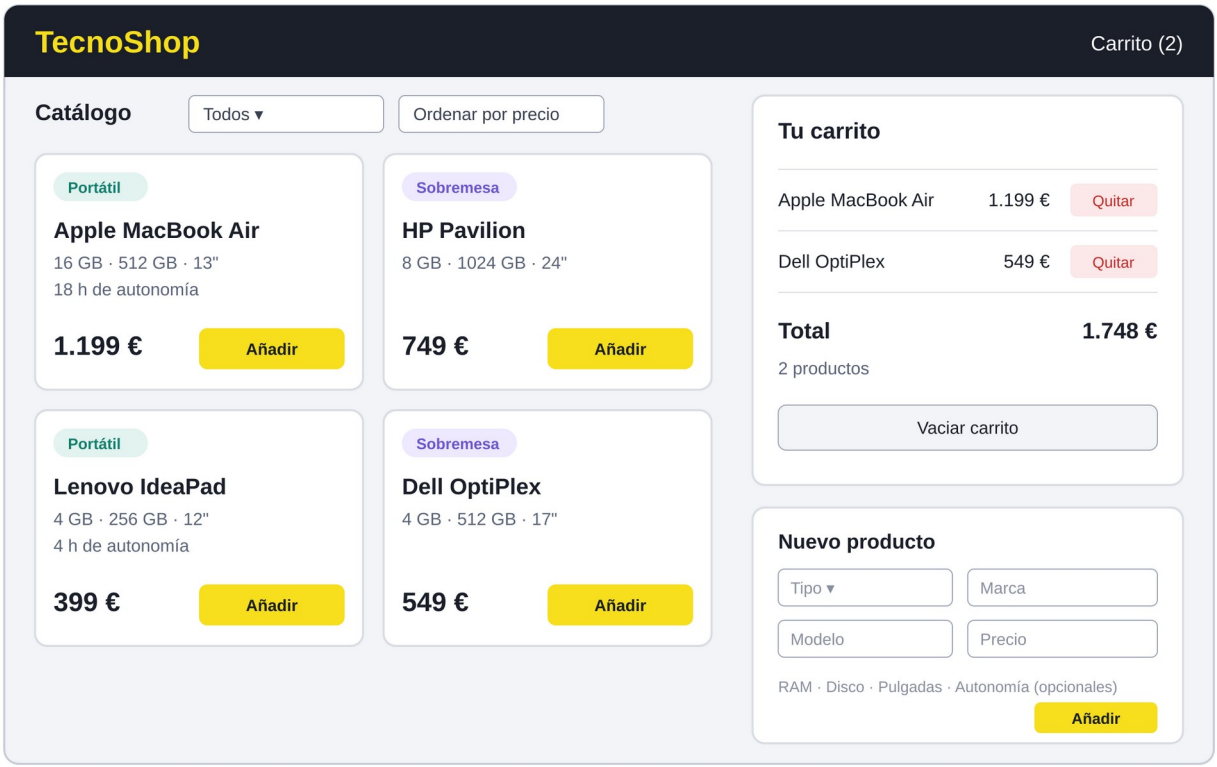


Figura 1. Aspecto orientativo de la aplicación.

Apartado 1. Clase Ordenador

Cada ordenador tiene estas propiedades:

Propiedad	Tipo	Valor por defecto
`marca`	texto	obligatoria
`modelo`	texto	obligatoria
`precio`	número (€)	obligatorio
`ram`	número (GB)	4
`disco`	número (GB)	512
`pulgadas`	número	17

1. Crea la clase con un constructor que reciba los valores en este orden: `marca, modelo, precio, ram, disco, pulgadas`. Usa **parámetros por defecto** para los tres últimos.
2. **Validación:** si `ram`, `disco` o `pulgadas` llegan con un valor menor o igual que 0, se asigna su valor por defecto. Si el precio no es mayor que 0, se guarda 0.
3. Añade un método `toString()` que devuelva los detalles en texto, por ejemplo: «HP Pavilion · 8 GB RAM · 1024 GB · 24"».
4. Añade un método `comparar(otro)` que reciba otro ordenador y devuelva «más barato», «más caro» o «mismo precio».

Apartado 2. Clase Portatil

1. Crea la clase `Portatil`, que **hereda** de `Ordenador` y añade la propiedad `autonomia` (en horas).
2. Se construye igual que un ordenador, pero con la autonomía al final: `marca, modelo, precio, ram, disco, pulgadas, autonomia`. Por defecto, un portátil tiene **4 GB de RAM, 256 GB de disco, 12 pulgadas y 4 horas** de autonomía.
3. Sobrescribe `toString()` para que añada la autonomía a los detalles. Reutiliza el método del padre con `super.toString()`.

Apartado 3. El catálogo en la página

1. Crea un array `catalogo` con al menos **seis productos**, mezclando sobremesas y portátiles. Para algunos indica todos los valores y para otros deja que se apliquen los valores por defecto.
2. Escribe una función `pintarCatalogo(productos)` que muestre cada producto como una tarjeta con: tipo (Sobremesa o Portátil, averiguado con `instanceof`), marca y modelo, detalles (`toString()`), precio y un botón «Añadir».
3. Igual que en la lista de la compra, la función recibe un array para poder pintar también el catálogo filtrado u ordenado.

Apartado 4. Clase Carrito

El carrito también es un objeto, con sus propios datos y operaciones:

1. Crea la clase `Carrito` con una propiedad **privada** `#productos` (un array, vacío al principio).
2. Métodos: `agregar(producto)`, `quitar(indice)`, `vaciar()`, `getProductos()` (devuelve una copia del array), `getCantidad()` y `getTotal()`, que calcula la suma de precios con `reduce`.
3. Al pulsar «Añadir» en una tarjeta, el producto se añade al carrito y se actualiza la vista del carrito: lista de productos con un botón «Quitar» cada uno, número de productos y total. Añade también un botón «Vaciar carrito».
4. Muestra el número de productos del carrito en la cabecera de la página: «Carrito (2)».

Pista: cada botón sabe qué producto es

Cuando crees el botón «Añadir» de una tarjeta, registra su evento en ese mismo momento: `boton.addEventListener("click", () => carrito.agregar(producto));`. La flecha recuerda el `producto` de esa tarjeta.

Apartado 5. Filtrar y ordenar

1. Añade un `<select>` con las opciones «Todos», «Sobremesa» y «Portátiles». Al cambiarlo, muestra solo los productos de ese tipo. Usa `filter` e `instanceof`.
2. Añade un botón «Ordenar por precio» que muestre el catálogo de menor a mayor precio, respetando el filtro elegido.

Cuidado con instanceof

Un portátil también es un ordenador, así que `portatil instanceof Ordenador` es `true`. Para quedarte solo con los de sobremesa, comprueba que **no** sean portátiles: `!(p instanceof Portatil)`.

Apartado 6. Dar de alta productos

1. Crea un formulario con: tipo (sobremesa o portátil), marca, modelo y precio (obligatorios), y RAM, disco, pulgadas y autonomía (opcionales). El campo de autonomía solo tiene sentido para los portátiles.
2. Al enviarlo (`submit` + `preventDefault`), crea un `Ordenador` o un `Portatil` según el tipo, añádelo al catálogo y vuelve a pintarlo. Si falta la marca, el modelo o el precio, muestra un mensaje de error y no lo añadas.
3. Los campos opcionales que se dejen vacíos deben tomar los valores por defecto de cada clase.

Pista: campos vacíos y valores por defecto

Un parámetro por defecto solo se aplica si llega `undefined`. Un campo vacío devuelve `""`, así que conviértelo antes: `const ram = campoRam.value === "" ? undefined : Number(campoRam.value);`

Apartado 7. Mejoras (opcional)

1. Botón «Exportar catálogo» que muestre en un `<pre>` el catálogo en formato JSON (`JSON.stringify(catalogo, null, 2)`).
2. Al añadir un producto que ya está en el carrito, en lugar de repetirlo, muestra «×2», «×3»… y tenlo en cuenta en el total.
3. En cada tarjeta, un pequeño texto que compare el producto con el más barato del catálogo usando `comparar()`.